	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering (PCCoE)** (An Autonomous Institute) Affiliated to Savitribai Phule Pune University(SPPU) ISO 21001:2018 Certified by TUV	

Course: Software Testing and Quality Assurance Laboratory	**Code:** BIT26PE11
Name: Amar Vaijinath Chavan	**PRN:** 124B2F001
Assignment 5: Implement an automated test using Selenium WebDriver and WebDriver Manager in Python/Java for a sample or real website, simulating a given scenario and verifying its success.	

1. OBJECTIVE

- To implement **Parameterization** by creating a modular function that accepts room details as arguments, eliminating redundant code.
- To integrate **Exception Handling** using try-except blocks to manage runtime errors such as ElementClickInterceptedException or NoSuchElementException.
- To verify the stability of the "Add Room" module by intentionally introducing an **Error Condition** (invalid data row) and ensuring the script recovers gracefully.

2. TEST ENVIRONMENT & CONFIGURATION

- **Operating System:** Windows 11 (XAMPP Environment).
- **Automation Tool:** Selenium WebDriver for Python.
- **Data Source:** External CSV file (roomdata2.csv) containing both valid and intentionally invalid (0,0,0) records.
- **Browser:** Google Chrome managed via WebDriverManager.

3. IMPLEMENTATION DETAILS

3.1 Parameterization Strategy

The script utilizes a parameterized function add_room_parameterized(driver, room_data). Instead of hardcoding values, it treats the room details as a dictionary object passed during the loop. This allows one single block of code to process an unlimited number of room entries from the CSV.

3.2 Exception Handling & Error Recovery

To demonstrate advanced testing, an **Error Condition** was introduced where a room name was set to "0" or left empty.

- **Detection:** The script uses a try block to monitor for ElementClickInterceptedException (which occurs if a previous modal didn't close) and ValueError (for invalid data).

- **Recovery Logic:** When an exception is caught, the script executes driver.refresh(). This forces the browser to reload the admin page, closing any stuck modals and resetting the UI state so the next test case can proceed.

4. TEST EXECUTION SCRIPT

```
import csv
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.common.exceptions import NoSuchElementException, ElementClickInterceptedException

# 1. PARAMETERIZED FUNCTION (Assignment 7 Core)
def add_room_parameterized(driver, room_data):
    try:
        # Check for Empty Data Logic (Manual Error Condition)
        if not room_data['name'] or room_data['name'].strip() == "" or room_data['name'] == "0":
            print(f"--- ERROR CONDITION: Invalid/Empty Room Name '{room_data['name']}' detected ---")
            raise ValueError("Invalid room name")

        print(f"Executing Test Case for: {room_data['name']}")

        # Trigger Modal
        driver.find_element(By.XPATH, "//button[@data-bs-target='#add-room']").click()
        time.sleep(1)

        # Fill details using parameters
        driver.find_element(By.NAME, "name").send_keys(room_data['name'])
        driver.find_element(By.NAME, "area").send_keys(room_data['area'])
        driver.find_element(By.NAME, "price").send_keys(room_data['price'])
        driver.find_element(By.NAME, "quantity").send_keys(room_data['quantity'])
        driver.find_element(By.NAME, "adult").send_keys(room_data['adult'])
        driver.find_element(By.NAME, "children").send_keys(room_data['children'])
        driver.find_element(By.NAME, "desc").send_keys(room_data['desc'])

        # Submit
        driver.find_element(By.XPATH, "//button[contains(text(), 'SUBMIT')]").click()
        time.sleep(3)
        print(f"SUCCESS: {room_data['name']} added.")
        return True

    # 2. EXCEPTION HANDLING: Catching ElementClickIntercepted and other errors
    except (NoSuchElementException, ValueError, ElementClickInterceptedException) as e:
        print(f"!!! EXCEPTION HANDLED !!!")
        print(f"Reason: {type(e).__name__} - {str(e)}")
        print(f"Status: Skipping '{room_data['name']}' and recovering...")
```

```
        return False

def main():
    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service)
    driver.set_window_size(1000, 1000)
    driver.set_window_position(900, 0)

    try:
        driver.get("http://localhost/hbwebsite/admin/index.php")
        driver.find_element(By.NAME, "admin_name").send_keys("admin")
        driver.find_element(By.NAME, "admin_pass").send_keys("admin")
        driver.find_element(By.NAME, "login").click()
        time.sleep(2)
        driver.get("http://localhost/hbwebsite/admin/rooms.php")

        with open("roomdata2.csv", 'r') as file:
            reader = csv.DictReader(file)
            for row in reader:
                success = add_room_parameterized(driver, row)
                if not success:
                    print("Action: Refreshing browser to clear blocked UI/Error state.")
                    driver.refresh()
                    time.sleep(2) # Give time for page to reload

                time.sleep(1)

    finally:
        print("--- Final Automation Report: Completed ---")
        driver.quit()

if __name__ == "__main__":
    main()
```

// data csv file with invalid data

name,area,price,*quantity,*adult,children,desc

premium_room_1,500,2000,*2,*2,2,Luxury premium room added during Assignment 7.

premium_room_2,550,2200,*2,*2,2,Luxury premium room added during Assignment 7.

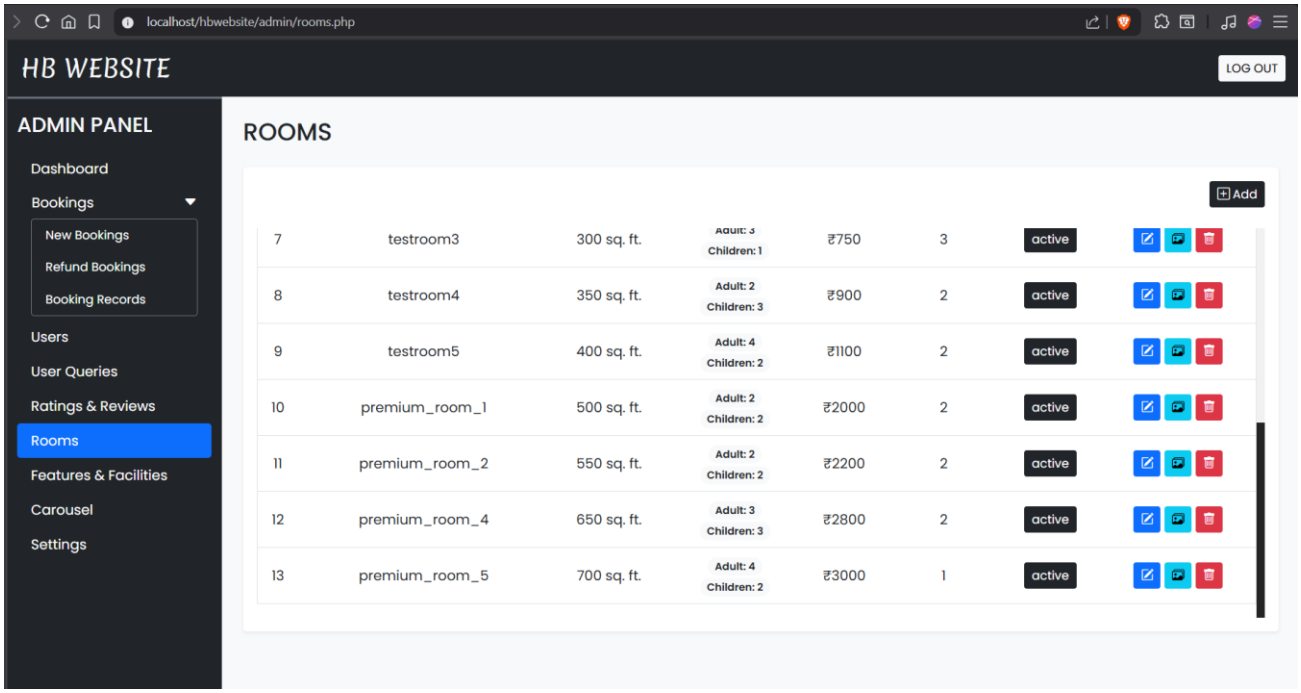
0,0,0,*0,*0,0,

premium_room_4,650,2800,*2,*3,3,Luxury premium room added during Assignment 7.

premium_room_5,700,3000,*1,*4,2,Luxury premium room added during Assignment 7.

5. TEST RESULTS & OBSERVATIONS

Test Case	Input Name	Expected Outcome	Actual Result	Status
TC_01	premium_room_1	Room Added	Room Added Successfully	PASS
TC_02	0	Exception Triggered	**ValueError Caught**; Page Refreshed	PASS
TC_03	premium_room_3	Room Added	Successfully added after recovery	PASS




```
[Running] python -u "c:\xampp\htdocs\hbwebsite\assignment7.py"
Executing Test Case for: premium_room_1
SUCCESS: premium_room_1 added.
Executing Test Case for: premium_room_2
SUCCESS: premium_room_2 added.
--- ERROR CONDITION: Invalid/Empty Room Name '0' detected ---
!!! EXCEPTION HANDLED !!!
Reason: ValueError - Invalid room name
Status: Skipping '0' and recovering...
Action: Refreshing browser to clear blocked UI/Error state.
Executing Test Case for: premium_room_4
SUCCESS: premium_room_4 added.
Executing Test Case for: premium_room_5
SUCCESS: premium_room_5 added.
--- Final Automation Report: Completed ---
```

6. CONCLUSION

Assignment No. 7 successfully demonstrates the implementation of a **Resilient Automation Framework**. By utilizing **Parameterization**, we achieved high code reusability. Furthermore, the integration of **Exception Handling** proved that the script could detect "Error Conditions," recover via a page refresh, and continue the test suite without manual intervention or program termination.